

장외 채권 OTC 고속 실행 엔진

Excel 전략 위임형 Windows 데스크톱 제어 시스템 보고서

보고 대상	채권상품부 · 비개발자 · 개발자
프로젝트 성격	장외 채권 OTC 메신저 판단·확정 루프 가속 Excel 전략 위임 + fetch/action 전담 Windows 엔진
작성 기준일	2026 년 8 월 24 일

목차

본문

- 1. 프로젝트 요약.....3
- 2. 문제 정의 및 해결 방식.....4
- 3. 업무 프로세스 및 시스템 개요.....6
- 4. 핵심 기능 및 운영 원칙.....8
- 5. 시스템 아키텍처.....11
- 6. 기술적 차별화 및 경쟁 우위.....13
- 7. 결론.....15

부록

- 부록 A. 모듈 및 Repository Map.....16
- 부록 B. 배포·실행 및 주요 설정.....17
- 부록 C. 오류·상태 정의.....18

1. 프로젝트 요약

핵심 결론

KBondWatcher 는 전략을 다시 만들지 않고, 기존 Excel 의 판단을 실전 OTC 메신저에서 목표 sub-second 실행으로 연결하는 Windows 데스크톱 제어 엔진이다.

장외 채권 시장에서는 브로커가 메신저에 올린 호가를 보고, 체결을 원할 경우 “ㅎㄷ”(확정) 등 확정 문장을 먼저 보내는 선점 경쟁이 발생한다. 수동 환경에서는 호가 확인, Excel key-in, 손익·베이스스 계산 확인, 메신저 입력까지 수초의 레깅이 생긴다.

KBondWatcher 는 이 수동 루프에서 반복되는 눈·손·키보드 작업을 자동화한다. 트레이더가 사전에 Profile 과 Excel 에 정의한 조건을 그대로 사용하며, 엔진은 전략을 새로 산출하지 않고 데이터 fetch, Excel 입력·읽기, UI action 만 담당한다.

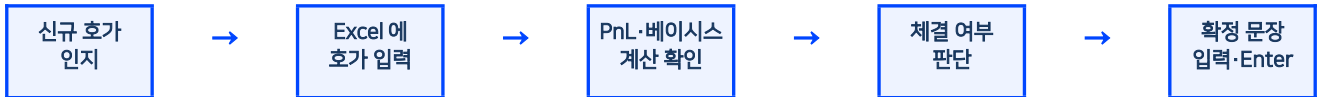
목표 sub-second	Excel 전략 유지	Local Blackbox	보수적 실행
호가 → Excel → UI 확정 1,000ms 미만 목표	기존 수식·시트 재사용 코드 포팅 불필요	워크북·수식 외부 업로드 없음	Fail-fast · One-shot Sanity guard

관점	한 줄 요약
도입·가치	기존 Excel 전략 그대로 · OTC 수동 레깅 제거 · 사전 조건의 고속 실행
운영·안전	가용성보다 오전송 방지 우선 · 로컬 블랙박스 · SENT 와 거래 성립 분리
기술	벤더 제공 거래·데이터 API 없이 Win32 / COM / UIA 로 이중 프로세스 연결
배포	Python·IDE 설치 불필요 · 로컬 웹 UI 에서 Profile / Calibration / START·STOP 제어

2. 문제 정의 및 해결 방식

2.1 장외 채권 OTC 거래와 수동 레깅

장외 채권 거래에서는 메신저 채팅방에 브로커가 호가를 올리고, 참가자가 확정 메시지를 보내 거래 의사를 표시한다. 원하는 종목·물량의 호가가 짧게 노출되는 상황에서는 다른 참가자보다 빠르게 확정하는 것이 중요하다.



2.2 수동 프로세스의 구조적 병목

병목	업무상 영향
선점 경쟁	1~3 초의 지연만으로도 호가가 사라지거나 다른 참가자가 먼저 확정할 수 있음
계산 레깅	현물 호가를 Excel에 직접 입력한 뒤 실시간 선물과 비교해야 하므로 즉시 반응하기 어려움
인지 부하	여러 종목·채팅방을 동시에 감시할수록 신규 호가 누락·오입력 위험 증가
반복 작업	호가 확인 → key-in → 계산 확인 → 타이핑이 매 거래 기회마다 반복

2.3 기존 대안의 한계

대안	한계
알고리즘 트레이딩 플랫폼	전략을 플랫폼 DSL/API로 재구현해야 하며 채팅형 OTC 메신저와 직접 연동하기 어려움
일반 RPA	OCR·좌표 클릭 의존 시 지연·오인식이 발생하고, 금융 메신저 본문이 UIA에 노출되지 않는 경우가 있음
Excel VBA 만	채팅 읽기·전송·다중 창 포커스 제어를 Excel 내부만으로 안정적으로 처리하기 어려움
전략 클라우드 업로드	트레이더 고유 수식·워크북을 외부로 올려야 해 IP·컴플라이언스 요구와 충돌 가능

2. 문제 정의 및 해결 방식

2.4 KBondWatcher 의 해결 구조

해결 원칙

전략은 Excel 에 그대로 두고, 조건은 사람이 사전 설정하며, 엔진은 그 조건의 fetch + action 만 자동화한다.



Before / After

구분	수동 프로세스	KBondWatcher
호가 감지	트레이더가 채팅창을 지속 확인	엔진이 신규 라인만 감시·diff
계산	호가를 수동 입력하고 결과 셀 확인	호가를 입력 셀에 기록하고 계산 완료 후 output 읽기
판단 조건	트레이더가 실시간으로 눈으로 확인	트레이더가 사전에 정의한 threshold / direction 사용
확정	문장 타이핑 후 Enter	조건 충족 시 UI 입력·Enter 1 회 수행
오류 대응	사람 판단에 의존	모호·불안전 상태는 전송보다 중단 우선

목표 latency 는 1,000ms 미만이다. Pilot 에서는 total_ms, excel_ms, send_ms 를 분리 기록하며, 목표와 실측을 구분해 관리한다. UI 전송 완료는 엔진의 입력 경로가 정상 종료되었다는 의미이며 상대방과의 거래 성립을 보장하지 않는다.

3. 업무 프로세스 및 시스템 개요

3.1 KBondWatcher 의 역할

KBondWatcher 는 시장 데이터로 전략과 주문을 생성하는 알고리즘 트레이딩 스택이 아니라, 사람이 사전에 정한 Excel 수식과 Profile 조건을 실전 메신저에서 빠르게 실행하는 데스크톱 제어 레이어다. 법적·규제 분류를 단정하는 표현이 아니라 제품 아키텍처상 역할의 차이이다.

구분	알고 트레이딩형 스택	KBondWatcher
조건·전략의 주체	시스템 코드·모델이 전략을 산출	사람이 Excel·Profile 로 조건·수식을 사전 결정
엔진 역할	전략·계산·실행을 하나의 스택에 수용	사전 조건의 fetch + action 만 자동화
전략 표현	코드·플랫폼 전용 DSL	기존 Excel 수식·시트
데이터 경로	공식 API·시장 피드가 일반적	이미 실행 중인 데스크톱 UI
출력	주문·체결 API	채팅 확정 문장 UI 입력
실패 시	전략별 재시도·슬리피지 정책	모호·불안전 상태는 즉시 ERROR

3.2 Excel 전략 위임 구조

엔진과 Excel 사이의 인터페이스는 입력 셀과 출력 셀 주소로 단순화된다. PnL, 베이스 등 계산 논리는 Excel 내부에 그대로 남고, 엔진은 호가에서 파싱한 값을 입력한 뒤 freshness 전제를 충족한 결과만 읽는다.

Input Cell	Excel Formula	Output Cell	Trigger
호가에서 파싱한 yield 기록	트레이더 기존 전략·수식 유지	PnL·basis 등 결과 읽기	Profile threshold 와 비교

3. 업무 프로세스 및 시스템 개요

3.3 트레이더와 엔진의 역할 분담

트레이더	KBondWatcher
Excel 전략 시트 작성·유지	열린 워크북에 COM 으로 셀 읽기·쓰기
종목·방향·임계값·메시지 규칙 결정	Profile 에 instrument, looking_for, threshold 등을 저장
Mode·전송 대상 선택	Mode preset 으로 소스·전송 창 결정
START / STOP	WATCHING / SENT / STOPPED / ERROR 상태 관리
필요 시 Admin 승인 정책 사용	서명·lease 검증 및 감사 이벤트 기록

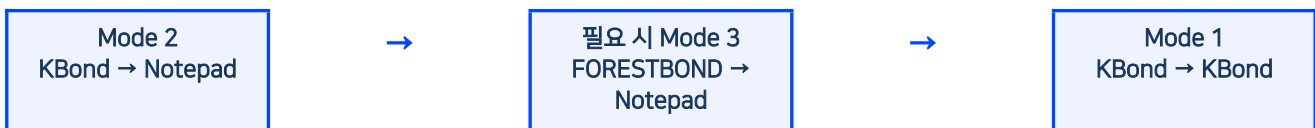
3.4 로컬 블랙박스

Excel 워크북과 수식은 트레이더 PC 로컬에만 존재하며 외부 업로드를 전제로 하지 않는다.

Admin 으로 전달되는 정보는 종목, Mode, 셀 주소, 메시지 템플릿 등 정책 메타데이터 중심이다.

Profile 은 승인 대상 Policy 와 현장 실행용 Runtime 필드로 분리해 통제와 사용 편의성을 함께 확보한다.

3.5 단계적 실전 투입



Mode 번호는 maturity 순서가 아니다. 권장 순서는 Mode 2 에서 KBond 파싱·Excel·트리거를 확인하고, UIA 경로가 필요한 경우 Mode 3 을 추가 검증한 뒤 Mode 1 에서 실전 채팅 전송을 수행하는 방식이다.

4. 핵심 기능 및 운영 원칙

4.1 End-to-End 실행 루프



단계	핵심 동작	안전 조건
Preflight	signed profile, lease, Excel bind, source/send window 확인	Calculation=Automatic assert, 대상 창 단 일성 확인
Source	신규 메시지 라인만 diff 후 Quote 파싱	한 poll 에 2 건 이상 매칭되면 ambiguous ERROR
Excel	input write → read-back → CalculationState DONE → output read	stale output 또는 disconnect 시 ERROR
Trigger	output 과 threshold 비교	sanity coarse guard 선행
Send	확정 문장 생성 후 foreground → click → paste → Enter	Mode 1 은 기본 one-shot 후 종료

4.2 Mode 1 · 2 · 3

Mode	소스(읽기)	전송(쓰기)	목적
1	KBond 분리창	같은 KBond 창	End-to-end 실전 Pilot
2	KBond 동일	메모장	파싱·Excel·트리거는 실전과 동일, 전송만 sandbox
3	FORESTBOND UIA Text	메모장	UIA 기반 source·diff 로직 검증

4. 핵심 기능 및 운영 원칙

4.3 보수적 운영 원칙 - “보내느니 멈춘다”

장외 확정 메시지는 주문에 준하는 행위다. KBondWatcher 는 끊임 없는 연속 가동보다 잘못된 확정·반복 확정의 위험을 낮추는 방향을 우선한다.

원칙	내용	의미
Fail-fast	판단이 모호하거나 결과가 불안전하면 fallback 없이 즉시 ERROR	추측에 의한 잘못된 확정 차단
One-shot	기본은 확정 메시지 1 회 후 즉시 종료	반복·대량 전송 가능성을 구조적으로 제한
Sanity band	극단적 numeric corruption 을 coarse guard 로 차단	정상 거래범위 검증이 아니라 garbage·오셀 값 방어
Freshness	Automatic assert, input read-back, calc DONE, stale guard	이전 계산값을 정상 output 으로 오인하지 않음

4.4 Fast-fail 과 재시도의 경계

분류	예	동작
Transient	Excel COM busy / RPC_E_SERVERCALL_RETRYLATER	제한적 retry 후 성공 시 계속, 한도 초과 시 ERROR
Ambiguous	한 poll 에 복수 매칭, 창 제목 ambiguous	즉시 ERROR - 어느 호가·어느 창인지 추측하지 않음
Unsafe	Excel disconnect, sanity 이탈, source/send 실패, HWND 무효	즉시 ERROR - 모르는 값으로 보내지 않음

4. 핵심 기능 및 운영 원칙

4.5 상태 머신과 Operational Boundary

상태	의미
WATCHING	호가 감시 및 조건 평가 중
SENT	foreground → click → clipboard → Enter 경로가 엔진 기준으로 예외 없이 끝난 상태
STOPPED	사용자 STOP 또는 정상 중지
ERROR	정상 전송 완료로 판정하지 못한 상태. 오류 시점에 따라 실제 UI 입력 여부가 불확실할 수 있음

단계	의미	KBondWatcher 보장 범위
SEND_ATTEMPT	전송 시도 audit 이벤트	시도 사실과 관련 로그 기록
UI 전송 완료 (SENT)	foreground → click → paste → Enter 코드 경로 완료	여기까지
거래 성립	브로커·상대방·시장 측 confirmation	보장하지 않음 - confirmation 시스템이 아님

4.6 Sanity band의 역할

```

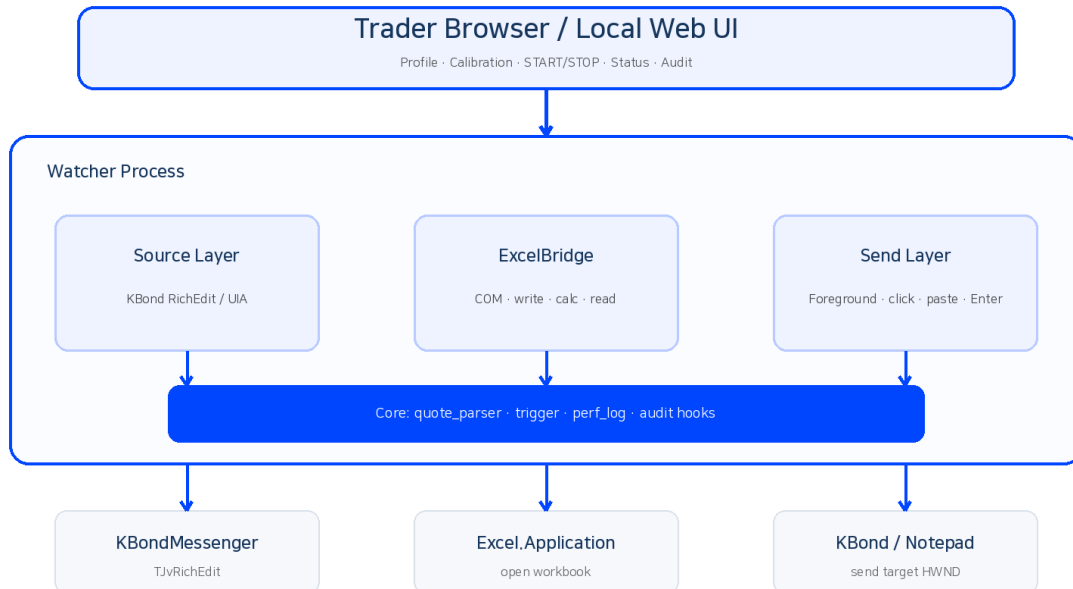
max_deviation = max(|threshold| × (sanity_band_pct / 100), sanity_abs_floor)
outside      ⇔ |output - threshold| > max_deviation
Default    : sanity_band_pct = 1000, sanity_abs_floor = 1e8

```

이 band는 전략상 정상 거래 범위를 검증하는 gate가 아니다. output 단위를 해석하지 않는 엔진이 수식 깨짐, 오셀 지정, 극단적 numeric corruption 등 자릿수 이상치를 잡기 위한 coarse guard다.

5. 시스템 아키텍처

5.1 전체 시스템 구조



KBondWatcher Pilot 아키텍처 - Browser / Watcher / Source / ExcelBridge / Send Layer

로컬 웹 UI는 Profile, Calibration, START/STOP, 상태·감사 조회를 제공한다. START 시 watcher process를 별도로 실행하며, watcher 내부의 Source Layer, ExcelBridge, Send Layer가 Core trigger·parser·audit 로직을 공유한다.

제어·상태는 %LOCALAPPDATA%/KBondWatcher/ 아래의 stop.flag, runtime_status.json, audit.jsonl, signed profile 등 파일 기반 신호를 사용한다.

5. 시스템 아키텍처

5.2 벤더 API 없는 이중 프로세스 연동

KBondWatcher 는 KBond·Excel 등이 제공하는 공식 거래·데이터 API 에 의존하지 않고, 이미 실행 중인 Windows 프로세스와 OS 수준 IPC 로 연결한다.

대상	메커니즘	기술적 의미
Excel	win32com GetObject("Excel.Application")	ROT 기반 COM Automation. 열린 인스턴스 bind, calc 상태·busy HRESULT·freshness 관리
KBond 채팅	EnumWindows → TjvRichEdit → EM_GETTEXTRANGE	UIA tree 에 본문이 없을 때 RichEdit Win32 message 로 텍스트 접근
RichEdit 본문	OpenProcess + VirtualAllocEx + ReadProcessMemory	대상 프로세스 주소 공간에 TEXTRANGE 구조체·UTF-16 버퍼를 할당해 cross-process read
FORESTBOND	pywinauto UIA Text enumeration	UIA 텍스트 노드 count watermark 기반 신규 라인 diff
전송	AttachThreadInput / foreground / ratio click / clipboard	Z-order·input focus 제어 후 paste + Enter
제어·상태	Local file IPC	stop.flag, runtime_status.json, audit.jsonl, signed profile

5.3 UIA 미노출 문제와 RichEdit 우회

KBondMessenger.exe 자식 창에서 TjvRichEdit 후보를 열거하고 채팅방 제목으로 단일 창을 특정한다.

EM_GETTEXTRANGE 와 원격 프로세스 메모리 할당을 이용해 전체 또는 꼬리 텍스트 스냅샷을 읽는다.

watermark·fingerprint 로 신규 라인만 diff 하며, 대상 창이 ambiguous 하면 즉시 ERROR 로 종료한다.

5. 시스템 아키텍처

5.4 Excel COM Bridge 및 Freshness

Freshness invariant

입력한 호가에 대응하는 최신 계산값이라는 확신이 없으면 output 을 거래 조건 판단에 사용하지 않는다.

단계	검증
1. START preflight	Excel Calculation == Automatic assert. 엔진이 사용자의 계산 설정을 임의 변경하지 않음
2. Input write	yield 입력 후 동일 셀을 read-back 하여 기록값 불일치 여부 확인
3. Calculation wait	Application.CalculationState == DONE 까지 대기한 후 output 읽기
4. Stale guard	input 이 바뀌었는데 output 이 불변이면 stale ERROR
외부 feed	RTD·외부 실시간 피드 자체의 지연은 별도 미보장

5.5 감시 루프

```

preflight → ExcelBridge.connect() → initialize_watermark
loop (~300ms):
  stop_flag? → STOPPED
  get_new_message_lines()
  0 match → continue / 1 match → quote / 2+ → ambiguous ERROR
  write_yield → read_back → calc DONE → output → sanity check
  evaluate(threshold) → format_message → send_text → SENT
  sent_after=exit → terminate (one-shot)

```

COM busy 는 Transient 로 제한적 재시도를 허용하지만, workbook disconnect 나 stale·sanity·ambiguous 상태는 재연결·추측 없이 즉시 종료한다.

6. 기술적 차별화 및 경쟁 우위

6.1 핵심 기술 차별화

포인트	설명
Excel 전략 재사용	전략 코드 포팅 없이 기존 수식·시트를 실행 레이어와 연결
Cross-process RichEdit read	UIA 미노출 금융 메신저를 Win32 message + 원격 메모리 IPC 로 우회
안전 상태머신	Ambiguous / Unsafe / Transient 를 분리하고 fail-fast·one-shot 으로 실행 범위 제한
Excel freshness	Automatic assert, input read-back, calc DONE, stale guard 로 계산값 신선도 확인
Watermark·fingerprint	신규 라인 diff 와 중복 처리 방어
Foreground hardening	AttachThreadInput, focus 복구, calibrated ratio click 로 UI send 경로 제어
Policy / Runtime 분리	승인 대상 필드와 현장 실행값을 분리해 운영 통제 구조 확보
Standalone 배포	Nuitka frozen build 로 Python·IDE 설치 없이 트레이더 PC 배포

6.2 기술 Moat 의 실질

장기 방어력은 저수준 Win32 API 호출 자체보다 OTC 워크플로 이해, 호가 파서와 edge case 추적, 안전 상태머신, Excel 전략 호환, 머신별 calibration·운영 계측이 겹치는 지점에 있다.

반대로 데스크톱 UI 자동화는 메신저 UI 변경, Windows 업데이트, 보안 SW 정책에 따른 유지보수 리스크를 가진다. Mode 분리, Calibration, smoke test, fail-fast 는 이 리스크를 없애는 것이 아니라 오전송보다 중단을 우선하도록 완충한다.

7. 결론

7.1 기존 방식 대비 경쟁 우위 요약

구분	전략 코드화 불필요	Excel 위임	Sub-second 목표	금융 메신저 실전
일반 RPA	○	△	△	×
알고 트레이딩형	×	×	○	△
Excel VBA 만	○	○	×	×
KBondWatcher	○	○	○	○

7.2 핵심 가치 요약

한 줄 정의

장외 채팅 호가를 읽고 → Excel 에 key-in 해 트레이더 수식으로 손익·베이스스를 평가하고 → 사전 설정 조건 충족 시 확정 문장을 UI 로 전송하는 Windows 전용 고속 실행 루프.



KBondWatcher 는 장외 시장에서 트레이더의 “머리와 손가락 사이”에 들어가는 엔진이다. 전략을 다시 만들지 않고 기존 Excel 의 판단을 사전 조건 기반의 고속 실행으로 연결하며, 엔진은 fetch 와 action 에 집중한다.

기술 기반은 RichEdit cross-process read, COM Excel bridge, UIA diff, 안전 상태머신이다. 동시에 UI·메신저 변화에 따른 유지보수를 전제로 하며, 정확한 계산·전송 경계가 확인되지 않으면 계속 동작하는 대신 중단하는 구조를 택한다.

부록 A. 모듈 및 Repository Map

경로	책임
main.py	CLI: --serve / --serve-admin / --run-profile, 배치 매칭
app/web/	Trader 로컬 웹 UI (FastAPI)
app/controller.py	Preflight, watcher subprocess spawn, Windows Job Object lifecycle
app/watcher_profile.py	Pilot 감시 메인 루프
app/profile.py	Policy vs Runtime 필드 및 검증
app/license.py	Device, DPAPI credential, signed lease
app/calibration.py	전송 창 클릭 좌표를 machine.json ratio 로 저장
source/richedit.py	TJvRichEdit 탐색, VirtualAllocEx 기반 텍스트 읽기
source/reader_kbond.py	Mode 1·2 KBond source reader
source/reader_uia.py	Mode 3 UIA source reader
source/quote_parser.py	호가 라인 파싱: instrument, yield, side, qty
excel/bridge.py	COM bind, Automatic assert, write/read-back, calc wait, stale guard
core/trigger.py	threshold 평가, side-flip token, 메시지 포맷
send/ui.py	foreground, ratio click, paste, Enter
admin/	정책 서명, lease, device 관리
build/build_nuitka.ps1	Standalone frozen 배포

부록 B. 배포·실행 및 주요 설정

B.1 트레이더 PC

```
main.exe --serve
Browser: http://127.0.0.1:8765/
START → main.exe --run-profile watcher spawn
```

B.2 Admin (선택)

```
set KBOND_ADMIN_URL=http://127.0.0.1:8770
main.exe --serve-admin
Device 등록 · signed profile 배포 · lease 갱신
```

B.3 주요 Profile / Runtime 항목

구분	대표 항목	의미
거래 대상	instrument, looking_for, required_qty	종목·방향·필요 수량
Excel	input_cell, pnl_cell	호가 입력 및 output read 위치
Trigger	threshold, threshold_op	사전 조건 평가 규칙
Message	message_template, confirm_token	확정 문장 포맷
Mode	mode, source/send preset	읽기·쓰기 대상 전환
Runtime	STOP 후 저장 가능한 실행값	현장 튜닝 값, 정책과 분리

B.4 Calibration

전송 대상 창에서 클릭 지점을 한 번 캡처해 machine.json 에 상대 좌표 ratio 로 저장한다. 해상도·DPI 차이를 완화하는 장치이며 UI 구조가 바뀌는 경우 재검증이 필요하다.

부록 C. 오류·상태 정의

C.1 주요 오류 분류

분류	대표 상황	원칙
Transient	COM busy / RPC_E_SERVERCALL_RETRYLATER	제한적 retry 후 한도 초과 시 ERROR
Ambiguous	복수 호가 매칭, 복수 동일 창, 대상 식별 불가	즉시 ERROR, 추측 금지
Unsafe	Excel disconnect, stale output, sanity fail, source/send 실패	즉시 ERROR

C.2 상태 정의

상태	정의	후속 동작
WATCHING	정상 감시 중	신규 메시지 polling 지속
SENT	UI send 코드 경로 정상 종료	기본 one-shot 이면 프로세스 종료
STOPPED	사용자 또는 정상 중지	재시작 전까지 비활성
ERROR	안전하게 계속 진행할 수 없는 상태	전송 완료로 간주하지 않고 즉시 중지

C.3 감사 및 성능 로그

로그	주요 기록
audit.jsonl	TRIGGER_RESULT, SEND_ATTEMPT, SEND_RESULT, ERROR 등 이벤트
sent_perf.csv	total_ms, excel_ms, send_ms 등 구간별 latency
runtime_status.json	WATCHING / SENT / STOPPED / ERROR 상태 및 실행 정보

운영 원칙

SENT 는 거래 성립 confirmation 이 아니다. ERROR 는 정상 전송 완료로 판정하지 못한 상태이며, 오류 시점에 따라 실제 UI 입력 여부가 불확실할 수 있다. 재가동 전 로그와 화면 상태를 확인한다.